

学号：202300130183	姓名：宋浩宇
实验名称：篇章连贯性分析实验——基于句子顺序与上下文一致性分析	
1. 实验目的	
1. 理解篇章分析的基本概念。 2. 掌握文本连贯性（coherence）的含义。 3. 学习判断句子顺序对语义的影响。 4. 理解上下文在语义理解中的作用。 5. 分析文本结构是否合理。	
2. 实验原理	
篇章分析关注多个句子之间的关系，而不仅仅是单个句子。 一个合理的文本通常具有： 语义连贯 逻辑清晰 顺序合理 例如： 小明起床了。 他去学校上课。 他完成了作业。 → 连贯 如果打乱顺序： 他完成了作业。 小明起床了。 他去学校上课。 → 不连贯 本实验通过分析句子顺序和上下文关系，判断文本是否连贯。	
3. 实验步骤与代码	
实验步骤： 1. 输入原始文本 2. 构造打乱顺序文本 3. 对句子进行分词 4. 计算相邻句子的相似度 5. 对比不同顺序下的结果 6. 判断文本连贯性 7. 分析结果	
代码： <pre>import random import re import jieba import numpy as np from sklearn.feature_extraction.text import TfidfVectorizer from sklearn.metrics.pairwise import cosine_similarity DATA_PATH = r"./exp-5.1-data.txt"</pre>	

```

RNG_SEED = 42
def read_paragraphs(path):
    """按空行分段, 每段为有序句子列表(一句一行)。"""
    with open(path, "r", encoding="utf-8") as f:
        raw = f.read()
        blocks = re.split(r"\n\s*\n", raw.strip())
        paragraphs = []
        for block in blocks:
            lines = [ln.strip() for ln in block.splitlines() if
ln.strip()]
            if lines:
                paragraphs.append(lines)
        return paragraphs
def sentence_to_tfidf_doc(sentence):
    return " ".join(jieba.lcut(sentence))
def adjacent_pair_similarities(sentences):
    """相邻句 TF-IDF 余弦相似度列表; 单句返回空列表。"""
    if len(sentences) < 2:
        return []
    docs = [sentence_to_tfidf_doc(s) for s in sentences]
    # 已用 jieba 按词空格拼接; 默认 token_pattern 要求长度>=2, 会丢
掉「他」「了」等单字,
    # 导致相邻句几乎无共现词、余弦恒为 0。按空格切分即保留 jieba 词
元。
    vec = TfidfVectorizer(analyzer=lambda s: s.split())
    mat = vec.fit_transform(docs)
    sims = []
    for i in range(len(sentences) - 1):
        a = mat[i : i + 1]
        b = mat[i + 1 : i + 2]
        sims.append(float(cosine_similarity(a, b)[0, 0]))
    return sims
def mean_or_none(values):
    if not values:
        return None
    return float(np.mean(values))
def print_analysis(title, sentences, sims):
    print(f"\n--- {title} ---")
    for i, s in enumerate(sentences, 1):
        print(f" [{i}] {s}")
    if not sims:
        print(" (少于两句, 无相邻句相似度)")
        return
    print(" 相邻句余弦相似度:")

```

```

for i, sim in enumerate(sims, 1):
    print(f"    句{i}-句{i + 1}: {sim:.6f}")
print(f"    平均分: {mean_or_none(sims):.6f}")
def heuristic_verdict(orig_mean, shuf_mean):
    if orig_mean is None or shuf_mean is None:
        return "样本过短, 无法比较。"
    if orig_mean > shuf_mean:
        return (
            "原序相邻句相似度平均更高, 在该启发式下更倾向判断为「更连贯」;"
            "相似度仅为辅助指标, 不能覆盖所有语篇现象。"
        )
    if orig_mean < shuf_mean:
        return (
            "乱序平均分不低于原序: 说明仅靠词重叠/相邻相似度不足以刻画连贯性, "
            "需结合因果、时序等更高层信息。"
        )
    return "两者平均分接近, 需结合更多线索判断连贯性。"
def main():
    random.seed(RNG_SEED)
    paragraphs = read_paragraphs(DATA_PATH)
    print("===== 篇章连贯性: 相邻句 TF-IDF 相似度 =====")
    print(f"数据文件: {DATA_PATH}    随机种子(乱序): {RNG_SEED}")
    for pi, orig in enumerate(paragraphs, 1):
        print(f"\n{'=' * 16} 段落 {pi} {'=' * 16}")
        shuffled = orig[:]
        random.shuffle(shuffled)
        orig_sims = adjacent_pair_similarities(orig)
        shuf_sims = adjacent_pair_similarities(shuffled)
        print_analysis("原序(连贯)", orig, orig_sims)
        print_analysis("乱序(打乱)", shuffled, shuf_sims)
        om = mean_or_none(orig_sims)
        sm = mean_or_none(shuf_sims)
        print("\n--- 对比结论(启发式) ---")
        if om is not None and sm is not None:
            print(f"    原序平均分: {om:.6f}    乱序平均分: {sm:.6f}")
            print(f"    {heuristic_verdict(om, sm)}")
if __name__ == "__main__":
    main()

```

4. 实验结果（图表与输出）

输出结果为：

===== 篇章连贯性：相邻句 TF-IDF 相似度 =====

数据文件: ./exp-5.1-data.txt 随机种子(乱序): 42

===== 段落 1 =====

Building prefix dict from the default dictionary ...

Dumping model to file cache C:\Users\23676\AppData\Local\Temp\jieba.cache

Loading model cost 0.341 seconds.

Prefix dict has been built successfully.

--- 原序（连贯） ---

[1] 小明起床了。

[2] 他去学校上课。

[3] 他完成了作业。

相邻句余弦相似度:

句 1—句 2: 0.102882

句 2—句 3: 0.249897

平均分: 0.176389

--- 乱序（打乱） ---

[1] 他去学校上课。

[2] 小明起床了。

[3] 他完成了作业。

相邻句余弦相似度:

句 1—句 2: 0.102882

句 2—句 3: 0.289451

平均分: 0.196166

--- 对比结论（启发式） ---

原序平均分: 0.176389 乱序平均分: 0.196166

乱序平均分不低于原序：说明仅靠词重叠/相邻相似度不足以刻画连贯性，需结合因果、时序等更高层信息。

===== 段落 2 =====

--- 原序（连贯） ---

[1] 天气很好。

[2] 我去公园散步。

[3] 我买了一本书。

相邻句余弦相似度:

句 1—句 2: 0.096188

句 2—句 3: 0.210786

平均分: 0.153487

--- 乱序（打乱） ---

[1] 我买了一本书。

[2] 我去公园散步。

[3] 天气很好。

相邻句余弦相似度:

句 1—句 2: 0.210786

句 2—句 3: 0.096188

平均分: 0.153487

--- 对比结论（启发式） ---

原序平均分: 0.153487 乱序平均分: 0.153487

两者平均分接近，需结合更多线索判断连贯性。

进程已结束，退出代码为 0

图表结果为:

无图表结果

5. 结果分析

1. 原始文本与打乱文本的连贯性有何差异？

在你这次跑出来的结果里，第一段原序相邻句相似度的平均分约为零点一七六，乱序约为零点一九六，乱序反而略高，说明仅凭当前这种词袋意义上的相邻相似度，并不能稳定地区分“更连贯”和“更乱”。第二段原序与乱序的平均分相同，都是约零点一五三，这是因为三句两两之间的相似度数值在打乱后只是换了一下相邻对的位置，平均数不变，读起来却已不是原来那种时间或话题推进顺序。所以从阅读体验上，原序通常更符合事件先后与指代衔接，而打乱会破坏这种线索，但数值指标未必总是跟着升高或降低。

2. 为什么句子顺序会影响文本理解？

汉语篇章里很多衔接靠时间顺序、因果顺序和代词所指等，读者会默认后一句是在前一句的语境里往前推进。顺序一变，同样的句子可能被理解成倒叙、话题跳跃或指代关系不清，推理成本上升，甚至会读出和作者原意不同的情节关系。因此顺序不只是排列问题，而是直接影响读者如何建立上下文。

3. 相邻句子的相似度是否可以反映连贯性？

可以当作一种粗糙的辅助信号，但不能单独当作标准。相邻句若共享较多词语或相关表述，相似度往往较高，有时与话题延续一致。但连贯性还依赖逻辑、指代、时态与常识，很多衔接并不靠词面重复。你实验中第一段乱序分数更高，第二段平均分数完全不变，都说明仅靠相邻 TF 余弦相似度会与人的连贯感受脱节，只能参考，需要结合其他分析。

4. 是否存在“语义相关但不连贯”的情况？

存在。例如若干句都在谈“读书”或“天气”，词面或向量上可能相近，但若中间缺少因果或转折交代，直接并置会显得跳跃。反过来，连贯的叙述也可能用代词或省略造成相邻句词面重复很少，相似度却不高，但仍读起来顺。因此语义相关与篇章连贯不是一回事。

5. 篇章分析相比句子分析增加了哪些挑战？

句子分析多关注单句内部成分与含义，篇章分析要在多句之间建立指代、省略、逻辑关系和

整体结构，需要跨句搜索先行语并处理歧义。顺序、文体与说话人视角也会改变解释。评价指标更难统一，像本实验里的相邻相似度只能刻画局部词面重叠，难以覆盖长距离依赖和语用推理，这些都是上升到篇章层面后额外要面对的问题。

6. 实验总结

本次实验从数据文件读入两段各三句的短文，对每段在保持原句集合不变的前提下打乱顺序，用 jieba 分词后基于 TF 与 IDF 得到句向量，计算相邻句之间的余弦相似度并比较原序与乱序下的平均分与逐对数值，程序正常跑通并打印对比与启发式结论文本。

LLM 使用情况：使用 LLM 辅助完成代码编写